

Natural Language Processing

9. Large Language Models

Paolo Torrioni

Fall 2025

Notice

Credits

The present slides are largely an adaptation of existing material, including slides by [Jurafsky & Martin](#), [Manning](#), [Potts & MacCartney](#), [Black & Mortensen](#), and others, to whom I am very grateful.

Downloading and sharing

A copy of these slides can be downloaded from [virtuale](#) and stored for personal use only. Please do not redistribute.

Table of Contents

- Large Language Models
- Decoding
- Pretraining LLMs
- Finetuning LLMs

Large Language Models

Conditional Generation

- Conditional generation is the task of generating text conditioned on an input piece of text (**prompt**)
- Long context key to effective conditional generation because LM can look back into the prompting text
- Simple task: left-to-right text completion

Autoregressive Text Completion

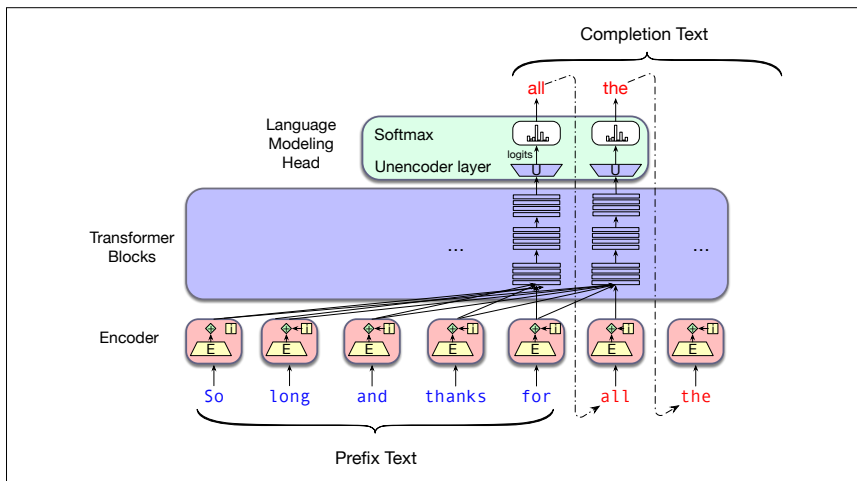


Figure 10.1 Left-to-right (also called autoregressive) text completion with transformer-based large language models. As each token is generated, it gets added onto the context as a prefix for generating the next token.

Conditional Generation

- Conditional generation is general
- Many practical NLP tasks can be cast as word prediction
- For example, **sentiment analysis**

The sentiment of the sentence ‘‘I like Jackie Chan’’ is

$P(\textit{positive} | \text{The sentiment [...] Jackie Chan'' is})$

$P(\textit{negative} | \text{The sentiment [...] Jackie Chan'' is})$

- **Question answering**

Q: Who wrote the book ‘‘The Origin of Species’’? A:

$\hat{y} = \underset{w}{\operatorname{argmax}} P(w | \text{Q: Who wrote [...]? A:})$

$\Rightarrow$ Charles

$\hat{y} = \underset{w}{\operatorname{argmax}} P(w | \text{Q: Who wrote [...]? A: Charles})$

Summarization

Original Article

The only thing crazier than a guy in snowbound Massachusetts boxing up the powdery white stuff and offering it for sale online? People are actually buying it. For \$89, self-styled entrepreneur Kyle Waring will ship you 6 pounds of Boston-area snow in an insulated Styrofoam box – enough for 10 to 15 snowballs, he says.

But not if you live in New England or surrounding states. “We will not ship snow to any states in the northeast!” says Waring’s website, ShipSnowYo.com. “We’re in the business of expunging snow!”

His website and social media accounts claim to have filled more than 133 orders for snow – more than 30 on Tuesday alone, his busiest day yet. With more than 45 total inches, Boston has set a record this winter for the snowiest month in its history. Most residents see the huge piles of snow choking their yards and sidewalks as a nuisance, but Waring saw an opportunity.

According to Boston.com, it all started a few weeks ago, when Waring and his wife were shoveling deep snow from their yard in Manchester-by-the-Sea, a coastal suburb north of Boston. He joked about shipping the stuff to friends and family in warmer states, and an idea was born. [...]

Summary

Kyle Waring will ship you 6 pounds of Boston-area snow in an insulated Styrofoam box – enough for 10 to 15 snowballs, he says. But not if you live in New England or surrounding states.

Figure 10.2 Excerpt from a sample article and its summary from the CNN/Daily Mail summarization corpus (Hermann et al., 2015b), (Nallapati et al., 2016).

Summarization

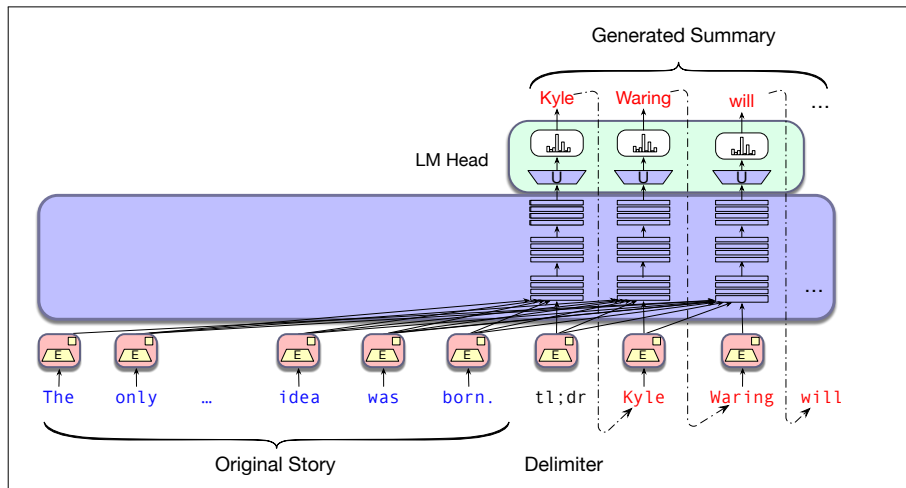
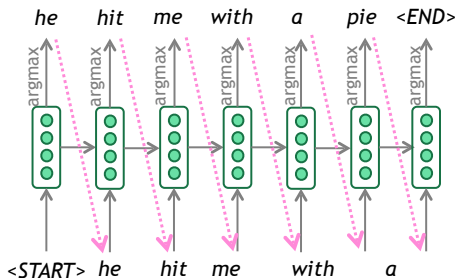


Figure 10.3 Summarization with large language models using the t1;dr token and context-based autoregressive generation.

Decoding

Greedy Decoding



- **Greedy decoding:** generate target sentence by taking argmax on each step of the decoder
- Deterministic
- **Problems** with this method?

Problem with Greedy Decoding

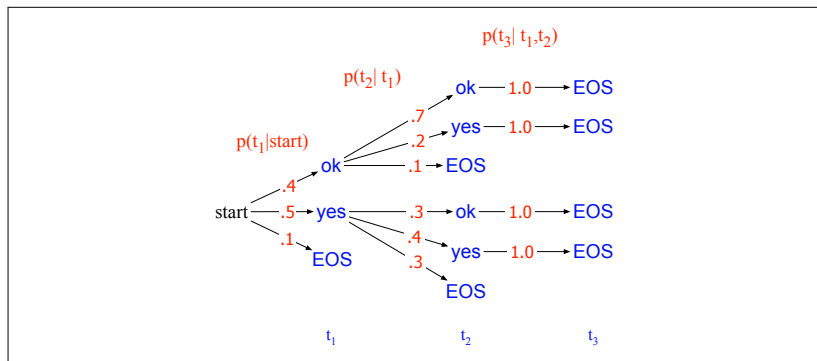


Figure 13.7 A search tree for generating the target string $T = t_1, t_2, \dots$ from vocabulary $V = \{\text{yes}, \text{ok}, \langle s \rangle\}$, showing the probability of generating each token from that state. Greedy search chooses *yes* followed by *yes*, instead of the globally most probable sequence *ok ok*.

- Is **exhaustive search** a viable alternative?

Beam Search

- **Beam search** enables to explore alternative decision sequences
- It uses a breadth-first search strategy with a heuristic filter that prunes the search frontier to stay within a fixed-size **beam width** or **size k**
- On each step of the decoder, keep track of the k “best” (most probable) output sequences (**hypotheses**)
- **Score hypotheses** using log probability
- Not optimal but much more **efficient** than the alternative

Beam Search Decoding Example

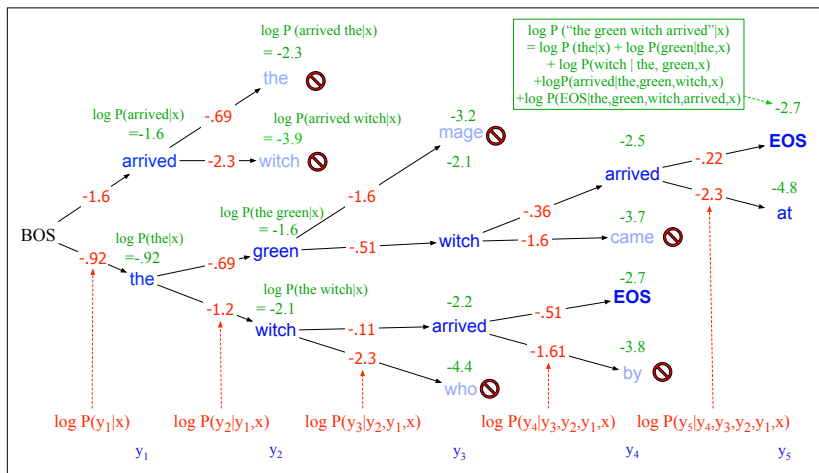


Figure 13.9 Scoring for beam search decoding with a beam width of $k = 2$. We maintain the log probability of each hypothesis in the beam by incrementally adding the logprob of generating each next token. Only the top k paths are extended to the next step.

Hypothesis Selection

- Different hypotheses may produce EOS token on **different timesteps**
- Need to apply a **length normalization** method

$$score(y) = \log P(y|x) = \frac{1}{t} \sum_{i=1}^t \log P(y_i | y_1, \dots, y_{i-1}, x)$$

- Beam search is popular in **machine translation**
 - May pass multiple options alongside with scores
 - or just most probable hypothesis

Sampling

- Decoding with LLMs in general has other objectives than MT
- Need to make outputs less predictable/repetitive: **sampling**
- For example, **random sampling**, seen with n-gram language models

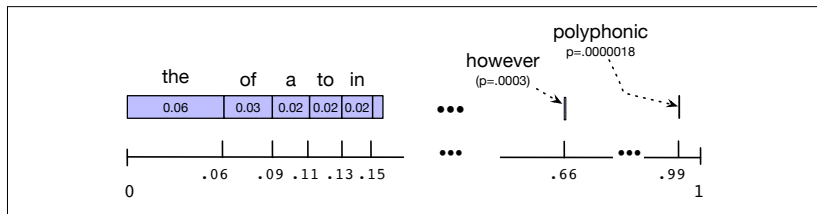


Figure 3.3 A visualization of the sampling distribution for sampling sentences by repeatedly sampling unigrams. The blue bar represents the frequency of each word. The number line shows the cumulative probabilities. If we choose a random number between 0 and 1, it will fall in an interval corresponding to some word. The expectation for the random number to fall in the larger intervals of one of the frequent words (*the*, *of*, *a*) is much higher than in the smaller interval of one of the rare words (*polyphonic*).

Sampling

- Random sampling mostly ok, but also **many** odd, rare, low-probability words in the tail
 - If added all up, large portion of the distribution
- Current LLMs use more sophisticated methods to avoid generating the very unlikely words
 - Top-k sampling
 - Top-p sampling
 - Temperature sampling
- Trade off between **quality** and **diversity**
 - Methods that emphasize the most probable words tend to produce generations that are rated by people as more accurate, more coherent, and more factual, but also more boring and more repetitive
 - Methods that give a bit more weight to the middle-probability words tend to be more creative and more diverse, but less factual and more likely to be incoherent or otherwise low-quality

Top-k Sampling

- ① Truncate the distribution to the top k most likely words
- ② Renormalize
- ③ Apply random sampling
 - If $k = 1$, top- k sampling is greedy decoding
 - The higher the k , the more diverse the text

Nucleus or Top-p Sampling

- A problem with top- k sampling is: k is fixed, but the shape of probability distribution varies with context
- A solution: consider not number of candidates, but **probability mass**

Definition (top- p vocabulary)

Given a distribution $P(w_t | \mathbf{w}_{<t})$, the **top- p vocabulary** $V^{(p)}$ is the smallest set of words such that

$$\sum_{w \in V^{(p)}} P(w | \mathbf{w}_{<t}) \geq p$$

- 1 Truncate the distribution to the top- p vocabulary
- 2 Renormalize
- 3 Apply random sampling

Temperature Sampling

- Don't truncate, but **reshape** the distribution
 - Higher temperature $\Rightarrow$ more flexibility, can explore more states
 - Lower temperature $\Rightarrow$ smoothly increase probability of most probable words, decrease that of rare words
- Implementation: divide logit by τ : $\mathbf{y} = \text{softmax}(u/\tau)$
 - Low-temperature sampling: $\tau \in (0, 1]$
 - As $\tau \rightarrow 0$ we approach **greedy decoding**
 - High-temperature sampling: $\tau > 1$

Pretraining Large Language Models

Pretraining Uses Self-Supervision and Teacher Forcing

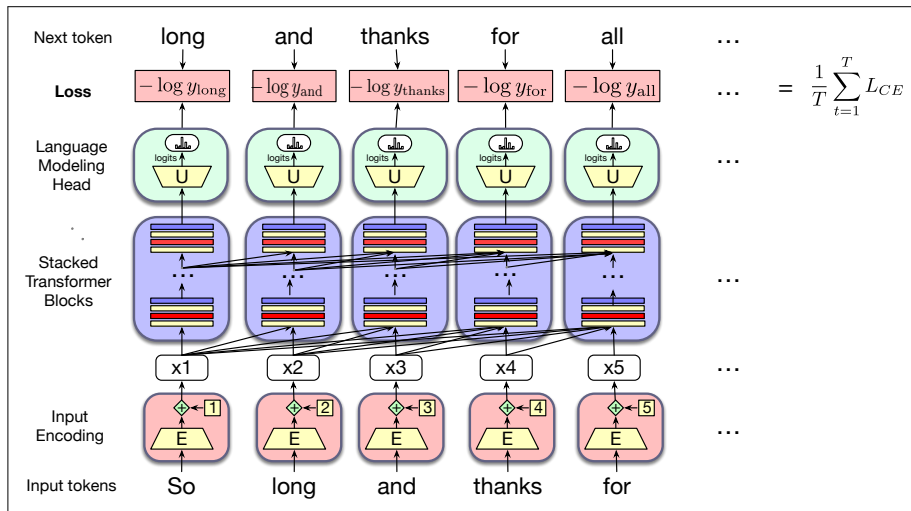


Figure 10.4 Training a transformer as a language model.

Pretraining Large Language Models

- Unlike RNN-based LMs, here calculation of losses isn't serial
- Usually full context window is filled, possibly by packing multiple documents separated by EOT
- Large batch size (10^6 tokens)
- Largely based on text scraped from the web, such as Wikipedia, books and news sites, augmented with curated data
- Some widely used training corpora
 - Versions of **Common Crawl** such as the **Colossal Clean Crawled Corpus** (C4): 156B tokens of English text
 - **The Pile**, 825 GB English text
 - **Dolma**, 3 trillion tokens

Composition of a Training Corpus

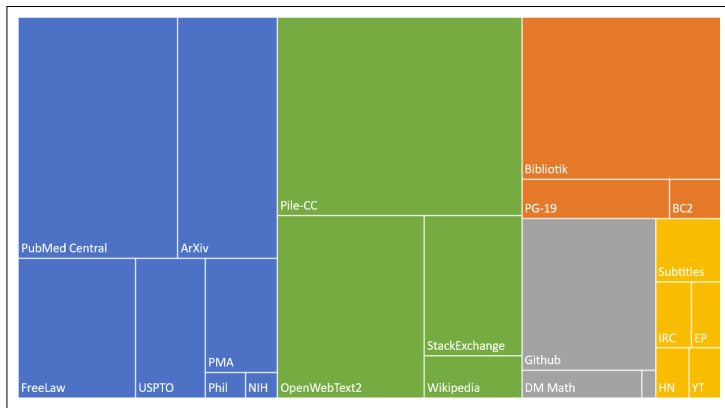


Figure 10.5 The Pile corpus, showing the size of different components, color coded as **academic** (articles from PubMed and ArXiv, patents from the USPTA; **internet** (webtext including a subset of the common crawl as well as Wikipedia), **prose** (a large corpus of books), **dialogue** (including movie subtitles and chat data), and **misc.** Figure from [Gao et al. \(2020\)](#).

Composition of a Training Corpus

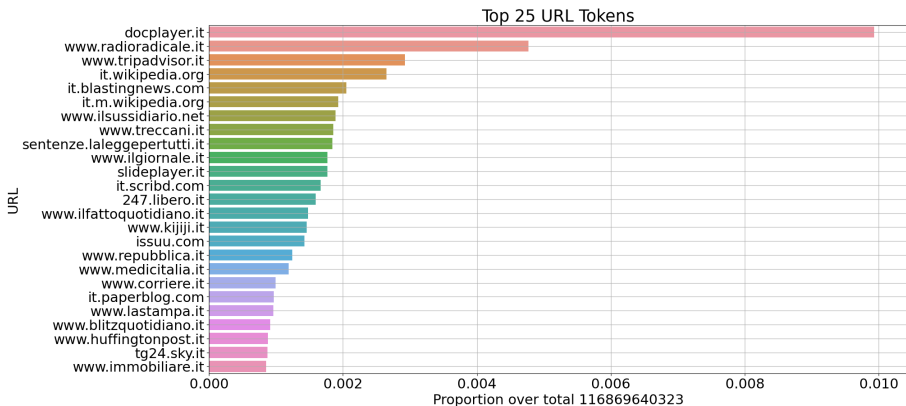


Image from <https://huggingface.co/sapienzanlp/Minerva-3B-base-v1.0>

Known Issues with Training Corpora

- Mostly English
- Quality is crucial (garbage in, garbage out)
 - Prefer high-quality reference corpora like Wikipedia, books, and particular websites to avoid PII or adult content
 - Remove boilerplate text
 - Deduplication at document, web page, or text level
- Safety filtering is also crucial
 - Toxicity classifiers (but high risk of false positives with minority dialects)
- Ethical and legal questions: is training LLMs **fair use** of **copyrighted** material? How about consent of website owners? Private information like IP addresses and phone numbers? (risk of **information leak**?)

How Much Training?

- Generally, increasing the number of parameters of LLMs (**model size**) brings better performance
- Also training them longer, and with more text (**training data**), improves performance
- As model size and training data increase, the cost of training increase
- What is the best way of using a certain **compute budget**, in terms of model size and training data?

Scaling Laws

- Scaling laws are empirical laws that express the relationships between model size, i.e., non-embedding parameters

$N \approx 2d n_{layer}(2d_{attn} + d_{ff})$, training data D , and compute budget C

- The performance of an LLM (the loss, L) scales as a power-law with each of these dimensions if the other two of them are held constant:

$$L(N) = \left(\frac{N_c}{N}\right)^{\alpha_N} \quad L(D) = \left(\frac{C_c}{D}\right)^{\alpha_D} \quad L(C) = \left(\frac{C_c}{C}\right)^{\alpha_C}$$

- It is very important to know how much data we need to add, when scaling up a model. For instance
 - A Chinchilla optimal model requires about 20 tokens per parameter
 - Llama3 70B was trained with about 200 tokens per parameter

Finetuning Large Language Models

Finetuning

- How to apply a generic LLM to a specialized domain, e.g., legal or medical, or a particular language, or task?

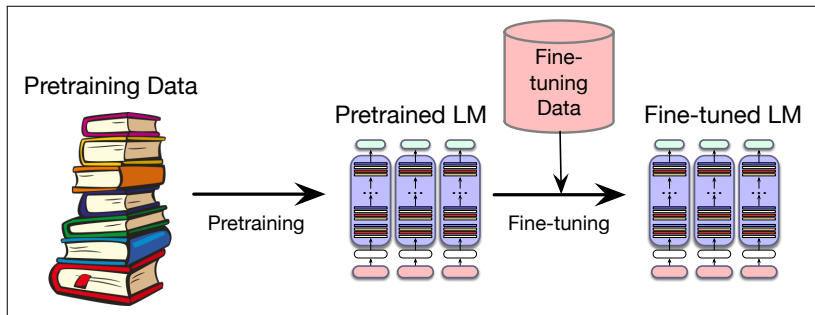


Figure 10.6 Pretraining and finetuning. A pre-trained model can be finetuned to a particular domain, dataset, or task. There are many different ways to finetune, depending on exactly which parameters are updated from the finetuning data: all the parameters, some of the parameters, or only the parameters of specific extra circuitry.

Finetuning

Four related kinds of LLM finetuning

- **Continued pretraining**: keep training all the model using domain-specific text
- **Parameter-efficient finetuning** or PEFT: select which parameters to update and keep the rest frozen
- Task-specific fine-tuning by adding a **head** after the top layer of the model, e.g., a classification head
- **Supervised finetuning**, for example for **instruction finetuning** to teach a model to follow instructions. Requires a dedicated dataset of prompts/responses

Parameter-Efficient Fine Tuning

- Fine-tuning can be prohibitively expensive with large models
- Each pass of batch gradient descent has to backpropagate through many layers
- Idea: select which parameters to update in order to obtain maximum effect with minimum computation, keep the rest frozen
- Very popular method is **Low-Rank Adaptation** or **LoRA**
 - Consider a matrix $\mathbf{W} \in \mathbb{R}^{d \times k}$ to be updated via gradient descent
 - Create a low-rank decomposition of $\mathbf{W}$ made of two matrices, $\mathbf{A} \in \mathbb{R}^{d \times r}$ and $\mathbf{B} \in \mathbb{R}^{r \times k}$
 - During fine-tuning, update $\mathbf{A}, \mathbf{B}$, and replace $\mathbf{W} + \Delta\mathbf{W}$ with $\mathbf{W} + \mathbf{AB}$
- Computationally efficient, and enables working with multiple specialized models that differ only by their low-rank adaptations

LoRA

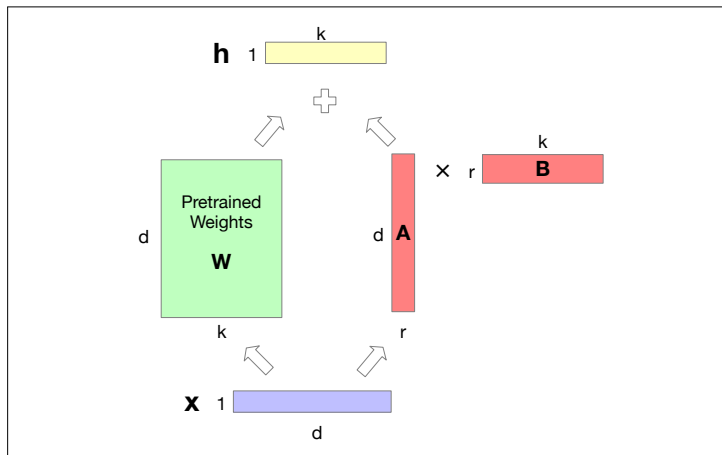


Figure 10.8 The intuition of LoRA. We freeze $\mathbf{W}$ to its pretrained values, and instead fine-tune by training a pair of matrices $\mathbf{A}$ and $\mathbf{B}$, updating those instead of $\mathbf{W}$, and just sum $\mathbf{W}$ and the updated $\mathbf{AB}$.

Questions?